Gimel Foundation

Gimel Foundation gGmbH i.G.
GiFo-Request for Comments: 0116
Obsoletes: -
Category: Standards Track
Version: 2.2

G. Wehberg, S. Raghuthaman and the Architecture Working Group of Gimel Foundation
8. April 2026

GAuth Interoperability

Abstract

This specification defines the interoperability layer for the Gimel Authorization (GAuth) protocol. It formalizes three components required for independent GAuth implementations:

- **Extended Token Format**: A JWT-based credential carrying Power of Attorney (PoA) attributes as defined in GiFo-RFC 0115, with a parallel W3C Verifiable Credential representation for eIDAS 2.0 / EUDI wallet interoperability.
- **PoA Credential Schema:** A canonical JSON Schema for the three-layer capability model (core verbs, permission-based capabilities, agent-based capabilities), serving as the single source of truth for both JWT claims and W3C VC credential subjects.
- **OAuth Engine Integration:** A standardized interface for integrating any OAuth 2.1 / OIDC-compliant authorization server as the token issuance engine for GAuth.

This RFC is a Gimel Foundation contribution paper intended for open-source publication under Apache License 2.0. It defines the protocol-level contracts that enable independent implementations to interoperate.

Proprietary platform extensions, i.e. so-called Exclusions (as per Legal Terms of Gimel Foundation) like AI-based governance, DNA-based identity and PQC associated, Web3 integration, are outside the scope of this specification and are not covered by the Apache 2.0 license (see Section 2 of this Request of Comment).

Status of This Memo

This is a Gimel Foundation Standards Track document.

This document is a product of the Gimel Foundation (GiFo). It represents the current consensus of the Gimel Foundation community. It has performed review and has been approved for publication.

Information about the status of this document, any errata, and how to provide feedback on it may be obtained at https://gimelfoundation.com or https://github.com/Gimel-Foundation.

Legal notice

Copyright (c) 2026 Gimel Foundation and the persons identified as the document authors. All rights are reserved.

This document is subject to the Gimel Foundation's Legal Provisions Relating to GiFo Documents (see http://GimelFoundation.com or https://github.com/Gimel-Foundation) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Code Components extracted from this document must include License text as described in Section 4. of the GiFo Legal Provisions Relating to Gimel Foundation Documents and are provided without warranty as described in the Provisions and its respective license conditions.

The distinguished GAuth standard as well as its interoperability standards are protected by copy right and patent law (patent pending). GAuth is an open-source standard, which can be based on OAuth, OpenID Connect and MCP. GAuth as well as these interoperability standards must not use Exclusions (see Scope under Section 2 of this Request for Comment), which are subject to separate license conditions and are also protected by copy right as well as patent law (patent pending).

Implementations of GAuth as well as these interoperability standards must reference to the Apache 2.0 license of OAuth and OpenID Connect as well as to the MIT license of MCP in line with its license conditions, as building blocks. Copyrights and licenses of these building blocks apply accordingly.

Implementations of GAuth as well as these interoperability standards must being licensed with Apache 2.0, granted by Gimel Foundation, and must not integrate Exclusions (as per Scope statement under Section 2 of this Request for Comment). Defined Exclusions of GAuth as well as of these interoperability standards must refer to separate license conditions.

Notational Conventions

The key words "Must", "Must Not", "Required", "Shall", "Shall Not", "Should", "Should Not", "Recommended", "May", and "Optional" in the following specification are to be interpreted as described in IETF`s RFC 2119.

* * *

Table of Content

1. Introduction

2. Legal Framework & Licensing

3. Architectural Overview

4. PoA Credential Schema - Canonical JSON Schema

5. Extended Token Format - JWT Claims

6. W3C Verifiable Credential Representation

7. OpenID4VC Integration - Issuance, Presentation & Selective Disclosure

8. OAuth Engine Integration

9. Adapter Types

10. Token Lifecycle

11. EUDI Wallet Compatibility

12. Security Considerations

13. Conformance Requirements

Appendices:

- A: GAuth PoA - W3C VC JSON-LD Context Definition
- B: GAuth PoA — JSON Schema (Canonical Format)
- C: Claims Registry
- D: OAuth Engine Comparison Matrix
- E: EUDI ARF Compliance Checklist
- F: Claims Enrichment Hook (Reference Implementation)
- G: Open Questions
- H: Reference Documents

1. INTRODUCTION

1.1 Purpose

GAuth (Gimel Authorization) is an open authorization protocol for governing agent-based software systems. It defines how agents receive, carry, and present authorization credentials — Power of Attorney (PoA) documents — that specify what an agent is supposed to do, under what constraints, and with whose authority.

GiFo-RFC 0110 defines the protocol engine. GiFo-RFC 0111 defines the authorization framework. GiFo-RFC 0115 defines the three-layer capability model (Vollmacht). This RFC - GiFo-RFC 0116 - defines the machine-readable serialization formats, the interoperability requirements, and the OAuth engine integration that enable independent implementations to interoperate.

1.2 Design Principles

- **Schema first, transport second** - The PoA data structure is defined as a canonical JSON Schema (Section 4). Transport formats (JWT, W3C VC, SD-JWT) are projections of this schema, not independent definitions.
- **Dual transport** - JWT for OAuth/OIDC-native environments; W3C Verifiable Credentials for eIDAS 2.0 / EUDI wallet environments. Same data, different envelopes.
- **Open protocol, protected intelligence** - The protocol, schemas, and OAuth engine interface are open-source (Apache 2.0). Platform-specific intelligence (AI governance, DNA identity, Web3) is proprietary and outside the scope of this specification (thus Exclusions as per the Legal Terms of Gimel Foundation).

1.3 Terminology

PoA (Power of Attorney / Vollmacht): A machine-readable authorization credential that delegates specific capabilities from a principal to an agent.

Extended Token: A signed JWT carrying PoA attributes as custom claims.

Adapter: A standardized interface implementation that integrates an external service with the GAuth runtime.

Credential Subject: The entity described by a Verifiable Credential - in GAuth, the PoA attributes.

Holder: The entity that possesses and presents a PoA credential (agent, wallet, or service).

Issuer: The GAuth Authorization Server that issues PoA credentials.

Verifier: A service that validates PoA credentials before granting permission or consent.

1.4 Relationship to Prior RFCs

```
GiFo-RFC 0110 ──→ Protocol Engine (transport, routing, service mesh)
    │
GiFo-RFC 0111 ──→ Authorization Framework (flows, roles, trust model)
    │
GiFo-RFC 0115 ──→ Three-Layer Capability Model (Vollmacht conceptual schema)
    │
GiFo-RFC 0116 ──→ Interoperability Layer (THIS DOCUMENT)
                  ├─ Canonical JSON Schema (Section 4)
                  ├─ JWT Extended Token (Section 5)
                  ├─ W3C VC Representation (Section 6)
                  ├─ OpenID4VC Integration (Section 7)
                  ├─ OAuth Engine Integration (Section 8)
                  └─ EUDI Wallet Compatibility (Section 11)
```

2. LEGAL FRAMEWORK & LICENSING

The following components of GiFo-RFC 0116 are published under the Apache License, Version 2.0:

In Scope (Open Source — Apache 2.0):

GAuth as well as these interoperability standards concern the technical field of AI and particularly the governance of AI. With the increasing prevalence and performance of AI, there is a growing need for effective control and governance mechanisms to ensure their safe and responsible use. GAuth together with these interoperability standards offer the solution for this. The GAuth standard together with these interoperability standards offer an AI control solution for digital agents, agentic teams of agents, humanoid robots and any other development of AI, which acts and decides on behalf of humans and/or

organizations, which are being represented by humans. It acknowledges that the ultimate accountability of such actions and/or decisions are with such humans and/or organizations, respectively.

That said, the scope of this GiFo-RfC 0116 includes:

- GAuth Protocol (RFCs 0110, 0111, 0115, 0116): Authorization protocol, capability model, token format, credential schema.
- PoA JSON Schema: Canonical data structure for PoA credentials (Section 4, Appendix B).
- JWT Extended Token Format: Claims structure and validation rules (Section 5).
- W3C VC Representation: JSON-LD context, credential schema, VC lifecycle (Section 6).
- OAuth Engine Adapter Interface: Standardized contract for AS integration (Section 8).
- Foundry Adapter Interface: Open interface for code execution environment integration (Section 9); concrete contract specified in the GAuth Implementation Specification.

For any further scope specification please refer to GiFo RFC0110, 0111 and 0115.

Out of Scope (Proprietary - Not covered by Apache 2.0, i.e. limitations on the right to make derivative works / Exclusions):

Users of GAuth as well as the standards presented in this GiFo-RfC 0116 Must Not – whether directly or indirectly – integrate:

- The use of Web3, i.e., blockchain technology (including respective web3 tokens and smart contracts),
- the use of AI or AI operators (e.g., trained models for governance, threat detection, structural compliance), i.e.,
  - AI that supports and/or controls the process of an AI deployment life cycle,
  - AI that tracks actions and/or decisions regarding authorization compliance,
  - AI that assures the quality of the outcome of AI engagements and
- DNA-based identities or identities based on genetic data, respectively, which offer particularly secure biometrics and/or seed for post-quantum cryptography
  - incl. AI that tracks the quality of DNA based identities
  - incl. AI that tracks risks in terms of identity theft or any other risks

For any further limitations and Exclusions please see GiFo RFC0110 and 0111.

In addition, this interoperability standards Must Not be used in other contexts than together with GAuth unless otherwise approved by Gimel Foundation in writing.

3. ARCHITECTURAL OVERVIEW

3.1 Open-Core Model

GiFo-RFC 0116 follows an open-core model. The protocol, data formats, and adapter interfaces for OAuth engine and foundry integration are fully open. Premium capabilities are provided through proprietary Gimel platform services that are outside the scope of this specification.

```
┌──────────────────────────────────────────────────────────────────┐
│  OPEN (Apache 2.0)                                               │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│  │ GAuth        │  │ PoA JSON     │  │ W3C VC       │          │
│  │ Protocol     │  │ Schema       │  │ Representation│         │
│  │ (RFC 0110-   │  │ (Section 4)  │  │ (Section 6)  │          │
│  │  0116)       │  │              │  │              │          │
│  └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
│  ┌──────────────────────────────────────────────┐               │
│  │         Open Adapter Interfaces               │              │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐     │              │
│  │  │ Type A   │  │ Type B   │  │ Type B   │     │              │
│  │  │ OAuth    │  │ Foundry  │  │ Wallet   │     │              │
│  │  │ Engine   │  │          │  │          │     │              │
│  │  └──────────┘  └──────────┘  └──────────┘     │              │
│  └──────────────────────────────────────────────┘               │
├──────────────────────────────────────────────────────────────────┤
│  PROPRIETARY (Gimel platform services — not in this RFC)         │
│                                                                  │
│  Type C: Premium extensions   │  Type D: Billing (internal       │
└──────────────────────────────────────────────────────────────────┘
```

3.2 Deployment Models

- **OAuth Engine**: Any OAuth 2.1 / OIDC AS via Type A adapter.
- **Foundry**: Any code execution environment via Type B adapter.
- **Governance**: Rule-based governance (open protocol, this specification) | *Optional proprietary extensions in terms of intelligent governance not in scope here due to Exclusions*.
- **Identity**: Standard OIDC Identity Provider | *Optional proprietary extensions in terms of DNA-based identity and/or PQC associated not in scope here due to Exclusions*.
- **Token Format**: Identical — RFC 0116 JWT / VC.
- **EUDI Wallet**: Full compatibility (open VC format).
- **Licensing**: Apache 2.0 (open protocol components only) | *Gimel Terms of Service for proprietary extensions, not in scope here*.

4. POA CREDENTIAL SCHEMA — CANONICAL JSON SCHEMA

4.1 Schema-First Architecture

The PoA credential schema is the single source of truth for all GAuth credential formats. Both the JWT Extended Token (Section 5) and the W3C VC representation (Section 6) are projections of this schema.

4.2 Three-Layer Capability Model

GiFo-RFC 0115 defines the Vollmacht (PoA) as a three-section document:

- Parties,
- Type & Scope,
- Requirements.

Within the scope, capabilities are organized into three layers:

```
┌──────────────────────────────────────────────────────────┐
│  PARTIES                                                 │
│  issuer, subject (agent), customer, project              │
├──────────────────────────────────────────────────────────┤
│  TYPE & SCOPE                                            │
│                                                          │
```

```
│  Layer 1: Core Verbs (Action Verbs)                    │
│  ┌──────────────────────────────────────────┐         │
│  │ foundry.file.create  │ foundry.file.modify │       │
│  │ foundry.file.delete  │ foundry.dependency.add│     │
│  │ foundry.command.run  │ foundry.agent.delegate│     │
│  └──────────────────────────────────────────┘         │
│                                                        │
│  Layer 2: Permission-Based Capabilities                │
│  ┌──────────────────────────────────────────┐         │
│  │ ToolPolicy per verb:                      │         │
│  │   allowed: boolean                        │         │
│  │   constraints: { pathPatterns, commands,  │         │
│  │     maxDelegationDepth, ... }             │         │
│  └──────────────────────────────────────────┘         │
│                                                        │
│  Layer 3: Agent-Based Capabilities                     │
│  ┌──────────────────────────────────────────┐         │
│  │ Platform Permissions:                     │         │
│  │   deployment: { targets, auto_deploy }    │         │
│  │   database: { read, write, migrate, prod } │        │
│  │   shell: { mode, allowlist/denylist }     │         │
│  │   packages: { verified_only }             │         │
│  │   external_apis: { allowed_domains }      │         │
│  │   secrets: { read, create }               │         │
│  └──────────────────────────────────────────┘         │
├──────────────────────────────────────────────────────┤
│  REQUIREMENTS                                          │
│  governance_profile, approval_mode, budget,            │
│  session_limits, ttl                                   │
└──────────────────────────────────────────────────────┘
```

4.3 Canonical JSON Schema

The full JSON Schema is provided in Appendix B. Here is the structure overview:

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json",
  "title": "GAuth PoA Credential",
  "description": "Canonical schema for GAuth Power of Attorney credentials (GiFo-RFC 0116)",
  "type": "object",
  "required": ["parties", "scope", "requirements"],
  "properties": {
    "schema_version": {
      "const": "0116.2.2"
    },
    "parties": {
      "type": "object",
      "required": ["issuer", "subject", "customer_id", "project_id"],
      "properties": {
        "issuer": { "type": "string", "format": "uri" },
        "subject": { "type": "string" },
        "customer_id": { "type": "string" },
        "project_id": { "type": "string" },
        "issued_by": { "type": "string" },
        "approval_chain": {
          "type": "array",
          "items": { "type": "string" },
          "description": "For four-eyes approval: ordered list of approver identities"
```

```
        }
      }
    },
    "delegation_chain": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["delegator", "delegate",
"scope_restriction"],
        "properties": {
          "delegator": { "type": "string", "description": "DID
or agent ID of the delegating entity" },
          "delegate": { "type": "string", "description": "DID
or agent ID of the receiving entity" },
          "scope_restriction": {
            "type": "object",
            "description": "Subset of scope that was delegated
(must be equal to or narrower than delegator's scope)"
          },
          "delegated_at": { "type": "string", "format": "date-
time" },
          "max_depth_remaining": { "type": "integer",
"minimum": 0 }
        }
      },
      "description": "Ordered chain of delegation from
original principal to current holder. Empty for direct-issue
credentials."
    },
    "scope": {
      "type": "object",
      "required": ["governance_profile", "phase",
"core_verbs"],
```

```
      "properties": {
        "governance_profile": {
          "type": "string",
          "enum": ["minimal", "standard", "strict",
"enterprise", "behoerde"]
        },
        "active_modules": {
          "type": "array",
          "items": { "type": "string" }
        },
        "phase": {
          "type": "string",
          "enum": ["plan", "build", "run"]
        },
        "allowed_paths": {
          "type": "array",
          "items": { "type": "string" }
        },
        "denied_paths": {
          "type": "array",
          "items": { "type": "string" }
        },
        "core_verbs": {
          "type": "object",
          "description": "Layer 1: Core action verbs with
Layer 2 permission policies",
          "additionalProperties": {
            "$ref": "#/$defs/ToolPolicy"
          }
        },
        "platform_permissions": {
```

```
          "$ref": "#/$defs/PlatformPermissions",
          "description": "Layer 3: Agent-based platform
capabilities"
        }
      }
    },
    "requirements": {
      "type": "object",
      "required": ["approval_mode"],
      "properties": {
        "approval_mode": {
          "type": "string",
          "enum": ["autonomous", "supervised", "four-eyes"]
        },
        "budget": {
          "$ref": "#/$defs/Budget"
        },
        "session_limits": {
          "$ref": "#/$defs/SessionLimits"
        },
        "ttl_seconds": {
          "type": "integer",
          "minimum": 60
        }
      }
    }
  },
  "$defs": {
    "ToolPolicy": {
      "type": "object",
```

```
      "required": ["allowed"],
      "properties": {
        "allowed": { "type": "boolean" },
        "cost_cents_base": { "type": "number", "minimum": 0 },
        "constraints": {
          "type": "object",
          "properties": {
            "path_patterns": { "type": "array", "items": {
"type": "string" } },
            "allowed_commands": { "type": "array", "items": {
"type": "string" } },
            "denied_commands": { "type": "array", "items": {
"type": "string" } },
            "max_delegation_depth": { "type": "integer",
"minimum": 0 },
            "max_file_size_bytes": { "type": "integer",
"minimum": 0 }
          }
        }
      }
    },
    "PlatformPermissions": {
      "type": "object",
      "properties": {
        "deployment": {
          "type": "object",
          "properties": {
            "targets": { "type": "array", "items": { "type":
"string", "enum": ["dev", "staging", "prod"] } },
            "auto_deploy": { "type": "boolean" }
          }
        },
```

```
        "database": {
          "type": "object",
          "properties": {
            "read": { "type": "boolean" },
            "write": { "type": "boolean" },
            "migrate": { "type": "boolean" },
            "production_access": { "type": "boolean" }
          }
        },
        "shell": {
          "type": "object",
          "properties": {
            "mode": { "type": "string", "enum": ["any",
"denylist", "allowlist"] },
            "allowlist": { "type": "array", "items": { "type":
"string" } },
            "denylist": { "type": "array", "items": { "type":
"string" } }
          }
        },
        "packages": {
          "type": "object",
          "properties": {
            "verified_only": { "type": "boolean" }
          }
        },
        "external_apis": {
          "type": "object",
          "properties": {
            "allowed_domains": { "type": "array", "items": {
"type": "string" } }
```

```
          }
        },
        "secrets": {
          "type": "object",
          "properties": {
            "read": { "type": "boolean" },
            "create": { "type": "boolean" }
          }
        }
      }
    },
    "Budget": {
      "type": "object",
      "properties": {
        "total_cents": { "type": "integer", "minimum": 0 },
        "remaining_cents": { "type": "integer", "minimum": 0 }
      }
    },
    "SessionLimits": {
      "type": "object",
      "properties": {
        "max_tool_calls": { "type": "integer", "minimum": 1 },
        "remaining_tool_calls": { "type": "integer",
"minimum": 0 },
        "max_lines_per_commit": { "type": "integer",
"minimum": 1 },
        "session_id": { "type": "string" },
        "started_at": { "type": "string", "format": "date-
time" }
      }
    }
```

```
  }
}
```

4.4 Scope Checksum

The `scope_checksum` claim in the JWT Extended Token provides tamper-evidence for the scope configuration. It is computed as:

```
scope_checksum = SHA-256(
  canonical_json(scope.governance_profile, scope.phase,
                 scope.allowed_paths, scope.denied_paths,
                 scope.active_modules,
                 tool_permissions_hash,
                 platform_permissions_hash)
)
```

Where `canonical_json` produces a deterministic JSON representation (sorted keys, no whitespace, UTF-8).

4.5 Permission Hashes

Full permission objects (`core_verbs`, `platform_permissions`) may be too large for the JWT size budget (Section 5.6). The JWT Extended Token carries hashes instead:

- `tool_permissions_hash = SHA-256(canonical_json(core_verbs))`
- `platform_permissions_hash = SHA-256(canonical_json(platform_permissions))`

Verifiers use stateful validation (Section 10.2.2) when they need the full permission set.

4.5.1 Core Verbs Reference

```
| Domain | Verb | `cost_cents_base` (example) | Resource URN |
|--------|------|---------------------------|-------------------
-|
| foundry | `foundry.file.create` | *(implementer-defined)* |
`urn:gauth:verb:foundry:file:create` |
```

| foundry | `foundry.file.modify` | *(implementer-defined)* | `urn:gauth:verb:foundry:file:modify` |

| foundry | `foundry.file.delete` | *(implementer-defined)* | `urn:gauth:verb:foundry:file:delete` |

| foundry | `foundry.dependency.add` | *(implementer-defined)* | `urn:gauth:verb:foundry:dependency:add` |

| foundry | `foundry.command.run` | *(implementer-defined)* | `urn:gauth:verb:foundry:command:run` |

| foundry | `foundry.agent.delegate` | *(implementer-defined)* | `urn:gauth:verb:foundry:agent:delegate` |

4.5.2 Platform Permissions Reference

| Domain | Permission | Default (Standard profile) | Resource URN |

|--------|-----------|---------------------------|---------------------|

| Deployment | targets | `["staging"]` | `urn:gauth:platform:deployment:targets` |

| Deployment | auto_deploy | `false` | `urn:gauth:platform:deployment:auto` |

| Database | read | `true` | `urn:gauth:platform:db:read` |

| Database | write | `false` | `urn:gauth:platform:db:write` |

| Database | migrate | `false` | `urn:gauth:platform:db:migrate` |

| Database | production_access | `false` | `urn:gauth:platform:db:prod` |

| Shell | mode | `allowlist` | `urn:gauth:platform:shell:mode` |

| Packages | verified_only | `true` | `urn:gauth:platform:packages:verified` |

| Secrets | read | `false` | `urn:gauth:platform:secrets:read` |

| Secrets | create | `false` | `urn:gauth:platform:secrets:create` |

4.5.3 Resource URN Scheme

Each permission is linked to a resource identifier using the following URN scheme:

```
urn:gauth:verb:<domain>:<action>        — Layer 1 core verbs
urn:gauth:platform:<domain>:<permission> — Layer 3 platform permissions
```

The resource URN is used by the policy evaluation engine (OPA/Cedar or equivalent) when evaluating access decisions. The engine maps the requested operation to a URN and checks the PoA's `core_verbs` and `platform_permissions` for the corresponding entry.

5. EXTENDED TOKEN FORMAT — JWT CLAIMS

5.1 Overview

The JWT Extended Token is the OAuth/OIDC-native transport for PoA credentials. It is a signed JWT (JWS) conforming to RFC 7519 and RFC 7515. The claims are a projection of the canonical JSON Schema (Section 4) into JWT claim namespaces.

5.2 Header

```json
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "ga_gov_cs_abc123"
}
```

Claim	Required	Values	Description

| `alg` | Yes | `RS256` (required), `ES256` (recommended) | Signing algorithm. `HS256` is prohibited (incompatible with multi-service trust) |

| `typ` | Yes | `JWT` | Token type |

| `kid` | Yes | Key ID from JWKS | References the signing key |

5.3 Standard Claims (RFC 7519)

| Claim | Type | Required | Description |

|---|---|---|---|

| `iss` | string | Yes | Issuer URI. Community: deployer's URI. Gimel-hosted: `https://gauth.gimel.dev` |

| `sub` | string | Yes | Subject — the agent this token authorizes |

| `aud` | string[] | Yes | Audience — services authorized to accept this token |

| `exp` | number | Yes | Expiration (Unix timestamp) |

| `iat` | number | Yes | Issued-at (Unix timestamp) |

| `nbf` | number | Yes | Not-before (Unix timestamp). Typically equals `iat` |

| `jti` | string | Yes | JWT ID. MUST be globally unique. Recommended format: `tok_` + UUIDv4 or equivalent high-entropy identifier. Implementations MAY use alternative formats provided global uniqueness is guaranteed (e.g., UUIDv7, ULID, or `{issuer_prefix}_{random}`). |

5.4 GAuth PoA Claims (`gauth` namespace)

The `gauth` namespace carries the PoA scope attributes from the canonical schema (Section 4). For agent mandates, the `gauth` and `gauth_mandate` namespaces represent two facets of the same credential: scope vs. operational state.

```json
{
  "gauth": {
    "version": "0116.2.2",
```

```
    "credential_id": "poa-cs-001-abc123",
    "customer_id": "cust_123",
    "project_id": "proj_456",
    "scope": {
      "governance_profile": "enterprise",
      "active_modules": ["security-basics", "data-model-
first"],
      "phase": "build",
      "allowed_paths": ["src/", "tests/"],
      "denied_paths": [".env", "secrets/"]
    },
    "scope_checksum": "sha256:a1b2c3d4...",
    "tool_permissions_hash": "sha256:4a5b6c7d...",
    "platform_permissions_hash": "sha256:1a2b3c4d...",
    "issued_by": "user_789",
    "approval_mode": "supervised"
  }
}
```

5.5 Mandate Claims (`gauth_mandate` namespace)

Present only when an agent mandate is active. Carries operational state for stateless validation.

```json
{
  "gauth_mandate": {
    "mandate_id": "mdt_abc",
    "mandate_status": "active",
    "budget": {
      "total_cents": 10000,
```

```
      "remaining_cents": 4200
    },
    "session": {
      "session_id": "sess_xyz789",
      "remaining_tool_calls": 87,
      "max_lines_per_commit": 100,
      "started_at": "2026-03-26T10:00:00Z"
    },
    "agent_capability_hash": "sha256:9f8e7d6c5b4a..."
  }
}
```

5.6 Token Size Budget

Constraint	Limit
GAuth Extended Token target	≤ 4 KB encoded
HTTP/2 SETTINGS_MAX_HEADER_LIST_SIZE	16 KB
nginx large_client_header_buffers default	8 KB per header

The token carries hashes of large objects (`tool_permissions_hash`, `platform_permissions_hash`) instead of the full objects. Full permission sets are resolved via stateful validation when needed. If a token would exceed 4 KB, the OAuth engine adapter MUST fail issuance.

6. W3C VERIFIABLE CREDENTIAL REPRESENTATION

6.1 PoA as Verifiable Credential

The GAuth PoA credential is represented as a W3C Verifiable Credential conforming to the [W3C Verifiable Credentials Data Model v2.0](https://www.w3.org/TR/vc-data-model-2.0/). This representation enables:

- eIDAS 2.0 compatibility (EUDI wallet storage and presentation)
- Cryptographic verification without a proprietary verification service

- Interoperability with the broader SSI (Self-Sovereign Identity) ecosystem

6.2 VC Structure

```json
{
  "@context": [
    "https://www.w3.org/ns/credentials/v2",
    "https://gimelfoundation.com/contexts/poa/v2"
  ],
  "id": "urn:gauth:credential:poa-cs-001-abc123",
  "type": ["VerifiableCredential", "GAuthPoACredential"],
  "issuer": {
    "id": "did:web:gauth.gimel.dev",
    "name": "GAuth Authorization Server"
  },
  "validFrom": "2026-03-26T10:00:00Z",
  "validUntil": "2026-03-26T11:00:00Z",
  "credentialSubject": {
    "id": "did:key:z6Mkf5rGMoatrSj1f4CyvuHBeXJEL...",
    "type": "GAuthAgent",
    "agentId": "foundry-claude-opus-4.6",
    "customerId": "cust_123",
    "projectId": "proj_456",
    "scope": {
      "governanceProfile": "enterprise",
      "activeModules": ["security-basics", "data-model-
first"],
      "phase": "build",
      "allowedPaths": ["src/", "tests/"],
      "deniedPaths": [".env", "secrets/"]
```

```
    },
    "coreVerbs": {
      "foundry.file.create": { "allowed": true,
"costCentsBase": 1 },
      "foundry.file.modify": { "allowed": true,
"costCentsBase": 1 },
      "foundry.file.delete": { "allowed": true,
"costCentsBase": 2 },
      "foundry.dependency.add": { "allowed": true,
"costCentsBase": 1 },
      "foundry.command.run": { "allowed": true,
"costCentsBase": 1 },
      "foundry.agent.delegate": { "allowed": false,
"costCentsBase": 5 }
    },
    "platformPermissions": {
      "deployment": { "targets": ["staging"], "autoDeploy":
false },
      "database": { "read": true, "write": false, "migrate":
false, "productionAccess": false },
      "shell": { "mode": "allowlist", "allowlist": ["npm",
"node", "tsc"] },
      "packages": { "verifiedOnly": true },
      "secrets": { "read": false, "create": false }
    },
    "requirements": {
      "approvalMode": "supervised",
      "budget": { "totalCents": 10000, "remainingCents": 8500
},
      "sessionLimits": {
        "maxToolCalls": 100,
        "remainingToolCalls": 87,
        "maxLinesPerCommit": 100
```

```
      }
    },
    "scopeChecksum": "sha256:a1b2c3d4..."
  },
  "credentialStatus": {
    "id":
"https://gauth.gimel.dev/credentials/status/3#12345",
    "type": "BitstringStatusListEntry",
    "statusPurpose": "revocation",
    "statusListIndex": "12345",
    "statusListCredential":
"https://gauth.gimel.dev/credentials/status/3"
  },
  "credentialSchema": {
    "id": "https://gimelfoundation.com/schemas/poa/v2.2/poa-
credential.json",
    "type": "JsonSchema"
  },
  "proof": {
    "type": "DataIntegrityProof",
    "cryptosuite": "ecdsa-rdfc-2019",
    "created": "2026-03-26T10:00:00Z",
    "verificationMethod": "did:web:gauth.gimel.dev#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z58DAdFfa9SkqZMVPxAQp..."
  }
}
```

6.2.1 VC Snapshot and Mutable Fields

The W3C VC representation is a snapshot of the credential at issuance time. Budget fields (`totalCents`, `remainingCents`) reflect the state at issuance and may become stale as the agent consumes budget. When a budget ceiling increase occurs (Section 10.3), a new W3C VC MUST be issued to reflect the updated budget. Verifiers using W3C VCs for budget validation SHOULD cross-reference the credential's `id` against the issuer's status list to confirm the credential has not been superseded.

6.3 JSON-LD Context

The GAuth PoA JSON-LD context defines the vocabulary for PoA-specific terms. The full context definition is in Appendix A.

Context URI: ` https://gimelfoundation.com/contexts/poa/v2`

The context maps the canonical schema property names to JSON-LD terms, enabling RDF-compatible processing while maintaining human-readable property names.

6.4 DID Methods

```
| Entity | DID Method | Example | Rationale |
|--------|-----------|---------|-----------|
| GAuth Service Endpoints | `did:web` | `did:web:gauth.gimel.dev` | DNS-verifiable, existing PKI, enterprise-friendly |
| Agents (ephemeral) | `did:key` | `did:key:z6Mkf5rGMoatrSj1f4...` | Self-certifying, no registration needed, disposable |
| Agents (persistent) | `did:web` | `did:web:foundry.gimel.dev:agents:opus-4.6` | For long-lived agent identities with discoverable DIDs |
```

6.5 Credential Status — Revocation

PoA revocation uses the [W3C Bitstring Status List v2.0](https://www.w3.org/TR/vc-bitstring-status-list/) specification:

- Each GAuth issuer maintains a status list credential at a well-known URL
- Each issued PoA credential is assigned a `statusListIndex` in the issuer's status list

- Revocation sets the corresponding bit to `1`
- Verifiers check the status list before accepting a PoA credential
- Status lists are cached (recommended TTL: 5 minutes) to balance latency and revocation propagation speed

Revocation reasons (encoded in revocation metadata):

- `agent_switch` — agent being replaced
- `session_end` — session completed normally
- `violation` — mandate terms violated
- `manual` — human-initiated revocation
- `budget_exceeded` — budget fully consumed

6.6 Dual Transport Strategy

```
| Context | Transport | Format | Use Case |

|---------|-----------|--------|----------|

| OAuth/OIDC-native services | JWT Extended Token (Section 5)
| JWS (signed JWT) | Service-to-service authorization, API
access |

| EUDI wallet | W3C VC (this section) | JSON-LD + Data
Integrity Proof | Wallet storage, cross-domain presentation |

| Selective disclosure | SD-JWT (Section 7.3) | SD-JWT |
Privacy-preserving presentation |

| Legacy REST | JWT in Authorization header | Bearer token |
Backward-compatible API access |
```

Both transports represent the same PoA credential (canonical schema), but with different fidelity. The JWT Extended Token carries scope metadata and permission hashes (for size constraints — see Section 5.6), requiring stateful lookup for full permission resolution. The W3C VC carries the complete credential subject including full `coreVerbs` and `platformPermissions` objects, enabling self-contained verification. The OAuth engine adapter issues both formats simultaneously when configured for dual-mode operation.

7. OPENID4VC INTEGRATION — ISSUANCE, PRESENTATION & SELECTIVE DISCLOSURE

7.1 OpenID4VCI — Credential Issuance

GAuth Authorization Servers issue PoA credentials via the [OpenID for Verifiable Credential Issuance (OpenID4VCI)](https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html) protocol. This enables standard wallets (including EUDI wallets) to receive GAuth PoA credentials.

7.1.1 Issuer Metadata

The GAuth AS exposes OpenID4VCI issuer metadata at `{issuer}/.well-known/openid-credential-issuer`:

```json
{
  "credential_issuer": "https://gauth.gimel.dev",
  "credential_endpoint": "https://gauth.gimel.dev/credentials",
  "credential_configurations_supported": {
    "GAuthPoACredential_jwt": {
      "format": "jwt_vc_json",
      "credential_definition": {
        "type": ["VerifiableCredential", "GAuthPoACredential"]
      },
      "cryptographic_binding_methods_supported": ["did:web", "did:key"],
      "credential_signing_alg_values_supported": ["ES256", "RS256"]
    },
    "GAuthPoACredential_sd_jwt": {
      "format": "vc+sd-jwt",
      "credential_definition": {
        "type": ["VerifiableCredential", "GAuthPoACredential"]
      },
      "cryptographic_binding_methods_supported": ["did:key"],
      "credential_signing_alg_values_supported": ["ES256"]
    }
```

```
  }
}
```

7.1.2 Issuance Flow

```
Agent / Wallet                        GAuth AS (Issuer)
    |                                     |
    |── Authorization Request ──────────→ |  (OAuth 2.0 / Pre-
Authorized Code)
    |←── Authorization Response ─────────|
    |                                     |
    |── Token Request ──────────────────→ |
    |←── Access Token ──────────────────|
    |                                     |
    |── Credential Request ─────────────→ |  (POST
/credentials)
    |   { format: "jwt_vc_json",          |
    |     types: ["GAuthPoACredential"], |
    |     proof: { ... } }                |
    |                                     |
    |←── Credential Response ───────────|
    |   { credential: "eyJhbGci..." }     |
```

For machine-to-machine flows (agent mandate issuance), the **Pre-Authorized Code Flow** is used. The mandate issuer provides a pre-authorized code that the agent exchanges for its PoA credential without user interaction.

7.2 OpenID4VP — Credential Presentation

Agents and wallets present PoA credentials to verifiers (services, PEP) via [OpenID for Verifiable Presentations (OpenID4VP)](https://openid.net/specs/openid-4-verifiable-presentations-1_0.html).

7.2.1 Presentation Flow

```
Agent / Wallet                    Verifier (Service / PEP)
    |                                   |
    |←── Presentation Request ──────────|
    |   { presentation_definition: {    |
    |       input_descriptors: [{       |
    |         constraints: {            |
    |           fields: [               |
    |             { path: "$.type",     |
    |               filter: { const:    |
    |                 "GAuthPoACredential"|
    |               }}                  |
    |           ]}                       |
    |       }]                          |
    |   }}                              |
    |                                   |
    |── Presentation Response ──────────→ |
    |   { vp_token: "eyJhbGci..." }     |
    |                                   |
    |←── Access Granted / Denied ──────|
```

7.2.2 Presentation Definition

Verifiers request specific PoA attributes via Presentation Exchange. For example, a service that needs to verify governance profile and budget:

```json
{
  "id": "gauth-poa-verification",
```

```
  "input_descriptors": [
    {
      "id": "poa-scope",
      "constraints": {
        "fields": [
          {
            "path":
["$.credentialSubject.scope.governanceProfile"],
            "filter": {
              "type": "string",
              "enum": ["enterprise", "strict", "behoerde"]
            }
          },
          {
            "path":
["$.credentialSubject.requirements.budget.remainingCents"],
            "filter": {
              "type": "integer",
              "minimum": 1
            }
          }
        ]
      }
    }
  ]
}
```

7.3 SD-JWT — Selective Disclosure

For privacy-preserving presentations, GAuth supports [SD-JWT (Selective Disclosure JWT)](https://datatracker.ietf.org/doc/draft-ietf-oauth-selective-disclosure-jwt/). This allows a PoA holder to disclose only the claims relevant to the verifier.

7.3.1 Disclosable Claims

```
| Claim Group | Always Disclosed | Selectively Disclosable |
|-------------|-----------------|------------------------|
| **Identity** | `sub` (agent ID) | `customer_id`, `project_id`, `issued_by` |
| **Scope** | `governance_profile`, `phase` | `active_modules`, `allowed_paths`, `denied_paths` |
| **Verbs** | — | Each individual verb permission |
| **Platform** | — | Each individual platform permission |
| **Budget** | — | `total_cents`, `remaining_cents` |
| **Session** | `session_id` | `remaining_tool_calls`, `max_lines_per_commit` |
```

7.3.2 SD-JWT Example

An agent presenting to a service that only needs to verify governance profile and phase (not the full permission set):

Issuer-signed JWT (with undisclosed claims hashed):

```
eyJhbGciOiJFUzI1NiJ9.eyJfc2QiOlsi...~
WyJkcGtJQkVJNmgiLCJjdXN0b21lcl9pZCIsImN1c3RfMTIzIl0~
WyI0Y0xYN3NiRjVwIiwicHJvamVjdF9pZCIsInByb2pfNDU2Il0
```

The verifier receives only the disclosed claims plus the SD-JWT signature. Non-disclosed claims are represented as hashes in the JWT — the verifier can confirm the JWT is validly signed without seeing the hidden claims.

7.4 JWKS and Key Discovery

GAuth issuers expose their public keys via two mechanisms:

```
| Mechanism | Endpoint | Use Case |
|-----------|---------|----------|
| JWKS (RFC 7517) | `{issuer}/.well-known/jwks.json` | JWT
token verification |
| DID Document | `did:web:{issuer-domain}` → DID Document |
W3C VC proof verification |
```

Both mechanisms reference the same underlying key material. The `kid` in the JWT header corresponds to the `id` of the verification method in the DID Document.

8. OAUTH ENGINE INTEGRATION

8.1 Adapter Interface Contract

The OAuth Engine adapter is the Type A adapter in the GAuth architecture (see Section 9 for the adapter taxonomy). Any OAuth 2.1 / OIDC compliant authorization server can be integrated via this interface.

The adapter interface (language-agnostic):

```
Interface: OAuthEngineAdapter


  issueToken(request: TokenIssuanceRequest) →
TokenIssuanceResponse
    Inputs:  subject, audience[], ttl_seconds, poa_claims
(from canonical schema)
    Outputs: signed_jwt, jti, kid, expires_at
    Errors:  INVALID_REQUEST, GOVERNANCE_REJECTED,
TOKEN_TOO_LARGE


  introspectToken(token: string) → TokenIntrospectionResponse
    Inputs:  raw JWT string
    Outputs: active (boolean), claims (parsed), remaining_ttl
    Errors:  INVALID_TOKEN, EXPIRED, REVOKED
```

```
  revokeToken(jti: string, reason: RevocationReason) → void
    Inputs:  JWT ID, reason enum
    Outputs: (none)
    Errors:  TOKEN_NOT_FOUND

  getJWKS() → JSONWebKeySet
    Inputs:  (none)
    Outputs: JWKS with all active public keys
    Errors:  (none)

  rotateKey(options: KeyRotationOptions) → KeyRotationResult
    Inputs:  algorithm preference, old_kid (optional)
    Outputs: new_kid, rotation_timestamp
    Errors:  ROTATION_IN_PROGRESS

  issueVerifiableCredential(request: VCIssuanceRequest) →
VCIssuanceResponse
    Inputs:  subject_did, poa_claims, format (jwt_vc_json |
vc+sd-jwt)
    Outputs: verifiable_credential, credential_id
    Errors:  INVALID_REQUEST, UNSUPPORTED_FORMAT
```

8.1.1 Error Response Schema

All adapter methods that return errors MUST use the following standardized error response structure:

```json
{
  "error": {
    "code": "GOVERNANCE_REJECTED",
```

```
    "message": "PoA issuance denied: governance profile
'behoerde' requires four-eyes approval",
    "details": {
      "field": "requirements.approval_mode",
      "expected": "four-eyes",
      "received": "autonomous"
    },
    "request_id": "req_abc123",
    "timestamp": "2026-03-26T10:00:00Z"
  }
}
```

Field	Type	Required	Description
`error.code`	string	Yes	Error code from the adapter method's error enum
`error.message`	string	Yes	Human-readable error description
`error.details`	object	No	Structured details specific to the error type
`error.request_id`	string	No	Correlation ID for tracing
`error.timestamp`	string	No	ISO 8601 timestamp of the error

Error codes (complete registry):

Code	HTTP Status	Description
`INVALID_REQUEST`	400	Malformed request or missing required fields
`GOVERNANCE_REJECTED`	403	Policy evaluation denied the operation

```
| `TOKEN_TOO_LARGE` | 413 | Resulting token would exceed the 4
KB size budget |

| `INVALID_TOKEN` | 401 | Token signature invalid or malformed
|

| `EXPIRED` | 401 | Token has expired (`exp` < current time) |

| `REVOKED` | 401 | Token has been revoked |

| `TOKEN_NOT_FOUND` | 404 | Referenced JTI does not exist |

| `ROTATION_IN_PROGRESS` | 409 | A key rotation is already in
progress |

| `UNSUPPORTED_FORMAT` | 422 | Requested credential format is
not supported by this engine |
```

8.2 OAuth Engine Comparison

The GAuth OAuth Engine Adapter interface (Section 8.1) is designed to be engine-agnostic. Any OAuth 2.1 / OIDC-compliant authorization server that supports custom claims injection into access tokens can serve as a GAuth engine. Appendix D provides a non-normative comparison matrix of commonly used engines.

The reference implementation uses Ory Hydra (Section 8.3). Implementations MAY use any engine listed in Appendix D or any other compliant engine not listed.

8.3 Reference Implementation: Ory Hydra

Hydra is the reference OAuth engine for GAuth due to its webhook-based token customization:

Before Hook — Called before token issuance. The GAuth runtime:

1. Validates the PoA request against the policy evaluation engine
2. Validates governance compliance (rule-based)
3. Returns `allow: true/false` to the OAuth engine

After Hook — Called after the engine generates the base JWT. The GAuth runtime:

1. Injects `gauth` and `gauth_mandate` claims
2. Computes and injects `scope_checksum`
3. Returns the enriched claims to the engine for signing

9. ADAPTER TYPES

9.1 Taxonomy

The GAuth architecture defines four adapter types. This specification (Apache 2.0) covers only Type A and Type B — the open interfaces that enable independent implementations.

```
| Type | Name | Interface | License | Examples |
|------|------|-----------|---------|----------|
| **A** | OAuth Engine | `OAuthEngineAdapter` (Section 8) | Open (Apache 2.0) | Ory Hydra, Keycloak, Auth0, Zitadel |
| **B** | Exchangeable Module | Open interface (per module) | Open (Apache 2.0) | Foundry (code execution), Wallet (credential storage) |
| **C** | Proprietary Extension | Proprietary | Gimel ToS | *(Outside scope of this specification)* |
| **D** | Internal | Gimel-internal | Not applicable | *(Outside scope of this specification)* |
```

9.2 Type A — OAuth Engine

The OAuth engine adapter has a fully open interface (Section 8). Any OAuth 2.1/OIDC-compliant authorization server can be plugged in.

9.3 Type B — Exchangeable Modules

Type B adapters provide open interfaces for services that users may swap with their own implementations. Reference implementations may be available from the Gimel platform. Implementations may also use any conformant third-party or self-hosted alternative.

- **Foundry** (Code Execution Environment): The foundry adapter provides the execution environment for agent operations. The open interface allows any compliant code execution environment to integrate with GAuth.
- **Wallet** (Credential Storage): The wallet adapter manages PoA credential storage and presentation. Compatible with EUDI wallets (Section 11) and other W3C VC-compliant wallet implementations.

9.4 Type C — Proprietary Extensions (Boundary)

Type C adapters provide proprietary extensions beyond the open protocol. They are not part of this specification. Implementations that use Type C adapters operate under separate licensing terms (see Section 2).

9.5 Type D — Billing (Boundary)

Type D adapters are internal to the Gimel platform and are not part of this specification.

10. TOKEN LIFECYCLE

10.1 Issuance Flow

```
Requesting Service                GAuth Runtime
  |                                    |
  |── Mandate/PoA Issue Request ──────→ |
  |                                    |
  |         ┌──────────────────────────────────────┐
  |         | 1. Validate request against PoA schema |
  |         | 2. Evaluate policy (rule-based)        |
  |         | 3. OAuth Engine Adapter: issue JWT     |
  |         |    (claims enrichment hook injects     |
  |         |     gauth claims from canonical schema) |
  |         | 4. (Optional) Issue W3C VC in parallel  |
  |         └──────────────────────────────────────┘
  |                                    |
  |←── { extended_token, vc (opt),     |
  |      scope_checksum, ... }         |
```

10.2 Validation Modes

10.2.1 Stateless Validation (Fast Path)

For read operations. Latency: <5ms.

1. Verify JWT signature against JWKS (`kid` → public key)
2. Check `exp` / `nbf` / `iat` temporal validity
3. Check `aud` contains the validating service's URI
4. Verify `scope_checksum` matches recomputed checksum
5. Check `governance_profile` permits the operation
6. If `gauth_mandate` present, check `mandate_status === "active"`

10.2.2 Stateful Validation (Full Path)

For write operations. Latency: ~50-100ms.

1. All stateless checks (10.2.1)
2. Mandate status lookup (active / revoked / expired / budget_exceeded / superseded)
3. Budget remaining > 0
4. `gauth.tool_permissions_hash` matches the hash computed from the stored mandate's tool permissions
5. Evaluate the specific operation against the full PoA using the policy engine

10.3 Renewal

Mandate scope is immutable — the capability set, governance profile, phase, paths, and approval mode cannot be changed after issuance. The only permitted in-session modification is a budget ceiling increase (additive-only), which triggers token re-issuance with updated budget claims. The previous token remains valid until its `exp` but carries stale budget data; verifiers using the stateful path (Section 10.2.2) will see the current budget from the mandate store. When operating in VC mode, a budget increase also requires re-issuance of the W3C VC (see Section 6.2.1).

10.4 Revocation

Mechanism	Trigger	Speed
Mandate revocation	API call	Immediate (stateful path)
VC Bitstring Status List	Status list update	Minutes (depends on cache TTL)
JWKS key rotation	Key compromise	Hours (JWKS propagation)

11. EUDI WALLET COMPATIBILITY

11.1 eIDAS 2.0 Architecture Reference Framework (ARF) Mapping

The GAuth PoA credential maps to the EUDI wallet ecosystem as follows:

ARF Concept	GAuth Mapping
Person Identification Data (PID)	Not applicable — GAuth authorizes agents, not persons. However, the `issued_by` field references the human principal's identity
Qualified Electronic Attestation of Attributes (QEAA)	GAuth PoA credential when issued by a Qualified Trust Service Provider (QTSP)
Electronic Attestation of Attributes (EAA)	GAuth PoA credential when issued by a non-qualified issuer
Wallet Instance	Agent or service holding the PoA credential
Wallet Provider	GAuth AS operator (community or Gimel-hosted)
Relying Party	Service verifying the PoA credential before granting access
Trusted List	Registry of GAuth issuers recognized by the trust framework

11.2 PoA as EUDI-Compatible Attestation

The GAuth PoA credential qualifies as an Electronic Attestation of Attributes (EAA) under eIDAS 2.0 when:

1. It is issued by a recognized GAuth Authorization Server
2. It conforms to the W3C VC Data Model v2.0 (Section 6)
3. It includes a `credentialStatus` pointing to a W3C Bitstring Status List (Section 6.5)
4. It is signed using an algorithm recognized by the ARF (ES256 recommended)
5. The issuer's DID Document is resolvable and contains the verification method

For **Qualified** EAA status (QEAA), the GAuth issuer must additionally: (a) be registered as a Qualified Trust Service Provider (QTSP) under a national eIDAS supervisory body, (b) meet the technical and organizational requirements of Articles 20-24 of the eIDAS Regulation, (c) use qualified electronic signatures or seals as defined in the Regulation, and (d) undergo conformity assessment by a conformity assessment body (CAB). The process is jurisdiction-specific and may take 6-12 months.

11.3 Credential Lifecycle in Wallet Context

```
1. ISSUANCE          Agent/Wallet ←── OpenID4VCI ──→ GAuth AS
                     Wallet stores PoA as W3C VC

2. STORAGE           Wallet holds PoA credential locally
                     Credential references Bitstring Status
List for revocation

3. PRESENTATION      Wallet ←── OpenID4VP ──→ Verifier
(Service/PEP)
                     Selective disclosure via SD-JWT
                     Verifier checks status list for
revocation
```

```
4. REVOCATION        GAuth AS updates Bitstring Status List

                     Bit at statusListIndex set to 1

                     Verifiers see revocation at next status
list check
```

11.4 Cross-Border Interoperability

A GAuth PoA credential issued by one GAuth instance is verifiable by another if:

1. The issuer's `did:web` DID is resolvable by the verifier
2. The issuer's public key is in the DID Document
3. The credential conforms to the GAuth PoA schema (Section 4)
4. The verifier trusts the issuer (directly or via a trust framework)

For EU cross-border scenarios, GAuth issuers can register in the **EU Trusted Lists** maintained by EU Member States under eIDAS Regulation (EU) 2024/1183.

11.5 Trust Establishment

```
| Mechanism | Scope | How GAuth Issuers Register |
|-----------|-------|--------------------------|
| **EU Trusted Lists** | Cross-border (EU) | Register as QTSP
under national supervisory body |
| **EBSI Trusted Issuers Registry** | EU institutional |
Register via EBSI API |
| **GAuth Federation** | GAuth ecosystem | Register in the
GAuth Issuer Registry at
`https://gimelfoundation.com/registry` |
| **Bilateral Trust** | Point-to-point | Verifier adds
issuer's DID to local trust store |
```

12. SECURITY CONSIDERATIONS

12.1 Token and Credential Security

Threat	Mitigation
Token/VC intercepted in transit	TLS 1.3 required for all GAuth communication
Token/VC stolen from storage	Short TTLs enforced. VCs SHOULD be stored in hardware-backed wallets
Token replay	`jti` enables replay detection. Validators SHOULD maintain a seen-set
Credential used by unauthorized service	`aud` (JWT) or Presentation Exchange constraints (VP) restrict valid verifiers
Scope manipulation	`scope_checksum` and `tool_permissions_hash` detect tampering

12.2 Signing Algorithms

Algorithm	Use Case	Status
`RS256` (RSA 2048+)	JWT Extended Token	Required support
`ES256` (ECDSA P-256)	JWT + W3C VC	Recommended (EUDI preferred)
`HS256` (HMAC-SHA256)	—	Prohibited (incompatible with multi-service trust)

12.3 Rate Limiting and Abuse Prevention

GAuth issuers and verifiers SHOULD implement rate limiting to protect against abuse:

Endpoint / Operation	Recommended Limit	Purpose
Token issuance (`issueToken`)	100 req/min per client	Prevent credential flooding

```
| Token introspection (`introspectToken`) | 1000 req/min per
verifier | Prevent introspection amplification |

| Status list endpoint | No rate limit (cacheable) | Public,
cached; set `Cache-Control: max-age=300` |

| JWKS endpoint | No rate limit (cacheable) | Public, cached;
set `Cache-Control: max-age=3600` |

| VC issuance (`issueVerifiableCredential`) | 50 req/min per
client | Prevent VC flooding |

| OpenID4VCI credential endpoint | 50 req/min per wallet |
Prevent wallet abuse |
```

Implementers MAY adjust these limits based on their deployment context. The limits above are recommended minimums for production deployments.

12.4 Audit and Logging Requirements

GAuth issuers MUST log the following events:

```
| Event | Required Fields | Retention |

|-------|----------------|-----------|

| Token/VC issuance | `jti`, `sub`, `iss`, `iat`, `exp`,
`governance_profile`, result (success/failure) | ≥ 90 days |

| Token/VC revocation | `jti`, reason, revoker identity,
timestamp | ≥ 1 year |

| Issuance rejection | Request details (sanitized), rejection
reason, policy rule | ≥ 90 days |
```

GAuth verifiers SHOULD log:

```
| Event | Required Fields | Retention |

|-------|----------------|-----------|

| Validation failure | `jti` (if available), failure reason,
verifier identity, timestamp | ≥ 30 days |
```

```
| Replay detection | `jti`, original presentation time, replay
attempt time | ≥ 90 days |
```

Log entries MUST NOT contain full token payloads or raw credential subjects, as these may contain sensitive scope information. Log the `jti`, `scope_checksum`, and outcome — not the credential content.

For deployments under the `behoerde` governance profile, audit log retention MUST comply with applicable public-sector record-keeping regulations (which may exceed the minimums above).

13. CONFORMANCE REQUIREMENTS

13.1 GAuth-Conformant Issuer

A GAuth-conformant issuer MUST:

1. Issue Extended Tokens conforming to Section 5 (JWT claims structure)

2. Support at least one credential format: JWT or W3C VC

3. Implement the OAuth Engine Adapter interface (Section 8.1)

4. Expose JWKS at `{issuer}/.well-known/jwks.json`

5. Compute `scope_checksum` per Section 4.4

6. Return errors conforming to the standard error response schema (Section 8.1.1)

7. Include a `schema_version` field in all issued credentials indicating the GiFo-RFC 0116 version

A GAuth-conformant issuer SHOULD:

1. Support both JWT and W3C VC credential formats

2. Support OpenID4VCI for credential issuance (Section 7.1)

3. Support SD-JWT for selective disclosure (Section 7.3)

4. Implement Bitstring Status List for revocation (Section 6.5)

5. Expose issuer configuration at `{issuer}/.well-known/gauth-configuration` (see Section 13.4)

6. Implement audit logging per Section 12.4

7. Implement rate limiting per Section 12.3

13.2 GAuth-Conformant Verifier

A GAuth-conformant verifier MUST:

1. Validate JWT signatures against the issuer's JWKS

2. Check standard claims (`exp`, `nbf`, `aud`, `jti`)

3. Verify `scope_checksum` integrity

4. Validate that `gauth.version` is a recognized version of this specification

A GAuth-conformant verifier SHOULD:

1. Support W3C VP verification via OpenID4VP

2. Check credential status via Bitstring Status List

3. Maintain `jti` seen-set for replay protection

4. Log validation failures per Section 12.4

13.3 Version Negotiation

The `gauth.version` claim (JWT) and `schema_version` property (canonical schema) carry the GiFo-RFC 0116 version in `MAJOR.MINOR` format (e.g., `"0116.2.2"`).

Versioning rules:

- **Minor version increments** (e.g., 2.1 → 2.2): Backward-compatible additions. Verifiers MUST accept tokens with any minor version they recognize. Verifiers SHOULD accept tokens with a higher minor version than they implement, treating unknown fields as non-critical.
- **Major version increments** (e.g., 2.x → 3.0): Breaking changes. Verifiers MAY reject tokens with a different major version. Issuers SHOULD support issuing tokens in the previous major version for a transition period of at least 6 months.

Migration guidance: When a new version of this specification is published, implementations SHOULD:

1. Accept credentials issued under the previous version for at least 6 months after the new version's publication date.

2. Issue credentials under the new version by default, with an option to issue under the previous version for backward compatibility.

3. Log version mismatches as informational events (not errors) to track migration progress.

13.4 GAuth Configuration Discovery

GAuth issuers SHOULD expose a discovery document at:

```
{issuer}/.well-known/gauth-configuration
```

Discovery document structure:

```json
{
  "issuer": "https://gauth.example.com",
  "gauth_version": "0116.2.2",
  "supported_versions": ["0116.2.1", "0116.2.2"],
  "jwks_uri": "https://gauth.example.com/.well-known/jwks.json",
  "credential_formats_supported": ["jwt", "w3c_vc", "sd-jwt"],
  "governance_profiles_supported": ["minimal", "standard", "strict", "enterprise", "behoerde"],
  "status_list_endpoint": "https://gauth.example.com/credentials/status",
  "openid4vci_endpoint": "https://gauth.example.com/credentials",
  "schema_uri": "https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json"
```

```
}
```

This enables verifiers and clients to discover the capabilities of a GAuth issuer programmatically, without prior configuration.

Appendix A: GAuth PoA — W3C VC JSON-LD Context Definition

```json
{
  "@context": {
    "@version": 1.1,
    "@protected": true,
    "gauth": "https://gimelfoundation.com/vocab#",
    "GAuthPoACredential": "gauth:GAuthPoACredential",
    "GAuthAgent": "gauth:GAuthAgent",
    "agentId": "gauth:agentId",
    "customerId": "gauth:customerId",
    "projectId": "gauth:projectId",
    "scope": "gauth:scope",
    "governanceProfile": "gauth:governanceProfile",
    "activeModules": {
      "@id": "gauth:activeModules",
      "@container": "@set"
    },
    "phase": "gauth:phase",
    "allowedPaths": {
      "@id": "gauth:allowedPaths",
      "@container": "@set"
    },
    "deniedPaths": {
      "@id": "gauth:deniedPaths",
      "@container": "@set"
    },
    "coreVerbs": "gauth:coreVerbs",
    "platformPermissions": "gauth:platformPermissions",
    "requirements": "gauth:requirements",
```

```
    "approvalMode": "gauth:approvalMode",
    "budget": "gauth:budget",
    "totalCents": "gauth:totalCents",
    "remainingCents": "gauth:remainingCents",
    "sessionLimits": "gauth:sessionLimits",
    "maxToolCalls": "gauth:maxToolCalls",
    "remainingToolCalls": "gauth:remainingToolCalls",
    "maxLinesPerCommit": "gauth:maxLinesPerCommit",
    "scopeChecksum": "gauth:scopeChecksum",
    "deployment": "gauth:deployment",
    "database": "gauth:database",
    "shell": "gauth:shell",
    "packages": "gauth:packages",
    "secrets": "gauth:secrets",
    "autoDeploy": "gauth:autoDeploy",
    "verifiedOnly": "gauth:verifiedOnly",
    "productionAccess": "gauth:productionAccess",
    "costCentsBase": "gauth:costCentsBase",
    "delegationChain": {
      "@id": "gauth:delegationChain",
      "@container": "@list"
    },
    "delegator": "gauth:delegator",
    "delegate": "gauth:delegate",
    "scopeRestriction": "gauth:scopeRestriction",
    "delegatedAt": "gauth:delegatedAt",
    "maxDepthRemaining": "gauth:maxDepthRemaining"
  }
}
```

Appendix B: GAuth PoA — JSON Schema (Canonical Format)

The complete JSON Schema is provided in Section 4.3 inline. The schema is published at:

Schema URI: `https://gimelfoundation.com/schemas/poa/v2.2/poa-credential.json`

Implementations MUST validate PoA credentials against this schema before issuance and upon receipt. The schema serves as the single source of truth for both JWT claims projection and W3C VC credential subject structure.

Appendix C: Claims Registry

C.1 `gauth` Namespace (PoA Scope Claims)

Claim Path	Type	Required	Validation
`gauth.version`	string	Yes	Regex: `^0116\.\d+\.\d+$`
`gauth.credential_id`	string	Yes	Non-empty
`gauth.customer_id`	string	Yes	Non-empty
`gauth.project_id`	string	Yes	Non-empty
`gauth.scope.governance_profile`	string	Yes	Enum: minimal, standard, strict, enterprise, behoerde
`gauth.scope.active_modules`	string[]	Yes	Valid module names
`gauth.scope.phase`	string	Yes	Enum: plan, build, run
`gauth.scope.allowed_paths`	string[]	Yes	Valid glob patterns
`gauth.scope.denied_paths`	string[]	Yes	Valid glob patterns
`gauth.scope_checksum`	string	Yes	Regex: `^sha256:[a-f0-9]{64}$`
`gauth.tool_permissions_hash`	string	Yes	SHA-256 format
`gauth.platform_permissions_hash`	string	Yes	SHA-256 format
`gauth.issued_by`	string	Yes	Non-empty

| `gauth.approval_mode` | string | Yes | Enum: autonomous, supervised, four-eyes |

C.2 `gauth_mandate` Namespace (Mandate State Claims)

| Claim Path | Type | Required | Validation |

|---|---|---|---|

| `gauth_mandate.mandate_id` | string | Conditional | Format: `mdt_*` |

| `gauth_mandate.mandate_status` | string | Conditional | Enum: active, expired, revoked, budget_exceeded, superseded |

| `gauth_mandate.budget.total_cents` | number | Conditional | ≥ 0 |

| `gauth_mandate.budget.remaining_cents` | number | Conditional | ≥ 0, ≤ total_cents |

| `gauth_mandate.session.session_id` | string | Conditional | Format: `sess_*` |

| `gauth_mandate.session.remaining_tool_calls` | number | Conditional | ≥ 0 |

| `gauth_mandate.session.max_lines_per_commit` | number | Conditional | > 0 |

| `gauth_mandate.session.started_at` | string | Conditional | ISO 8601 |

| `gauth_mandate.agent_capability_hash` | string | Conditional | SHA-256 format |

Appendix D: OAuth Engine Comparison Matrix

Non-normative. This comparison reflects the state of these projects as of the publication date. Implementations should verify current capabilities before selecting an engine.

| Criterion | Ory Hydra | Keycloak | Auth0 | Zitadel | Authelia | Dex |

|---|---|---|---|---|---|---|

| **License** | Apache 2.0 | Apache 2.0 | Proprietary | Apache 2.0 | Apache 2.0 | Apache 2.0 |

| **Self-Hostable** | Yes | Yes | No | Yes | Yes | Yes |

| **Custom Claims** | Before/After Hooks | Protocol Mapper SPI | Actions | Actions (JS) | Limited | None |

| **OIDC Compliant** | Full | Full | Full | Full | Partial | Partial |

| **VC Issuance** | Via hooks | Via SPI | Via Actions | Limited | No | No |

| **Resource Footprint** | Low (Go) | High (Java/JBoss) | N/A (SaaS) | Low (Go) | Very Low (Go) | Very Low (Go) |

| **User Management** | No (separate IdP) | Built-in | Built-in | Built-in | No | No |

| **Multi-Tenancy** | Via Hydra contexts | Via realms | Via tenants | Native | No | No |

Appendix E: EUDI ARF Compliance Checklist

| # | ARF Requirement | GAuth RFC 0116 Coverage | Status |

|---|---|---|---|

| 1 | Credential format: W3C VC Data Model 2.0 or SD-JWT VC | Section 6 (W3C VC), Section 7.3 (SD-JWT) | Covered |

| 2 | Credential status: Bitstring Status List | Section 6.5 | Covered |

| 3 | Issuance protocol: OpenID4VCI | Section 7.1 | Covered |

| 4 | Presentation protocol: OpenID4VP | Section 7.2 | Covered |

| 5 | Selective disclosure: SD-JWT | Section 7.3 | Covered |

| 6 | Cryptographic algorithms: ES256 (ECDSA P-256) | Section 12.2 (ES256 recommended) | Covered |

| 7 | DID methods for issuer identification | Section 6.4 (did:web) | Covered |

| 8 | Trust framework: EU Trusted Lists | Section 11.5 | Covered |

| 9 | Credential schema: JSON Schema validation | Section 4.3, Appendix B | Covered |

| 10 | Holder binding: proof of possession | Section 7.1.2 (credential request proof) | Covered |

| 11 | Revocation propagation: ≤ 24h for non-critical | Section 6.5 (5min recommended cache TTL) | Exceeds |

| 12 | Offline verification capability | JWT signature verification is offline-capable | Covered |

Appendix F: Claims Enrichment Hook (Reference Implementation)

F.1 Before Hook — Pre-Issuance Validation (Pseudocode)

```
function before_token_hook(oauth_request):
    subject = oauth_request.subject

    // 1. Look up the active PoA for this subject
    poa = lookup_active_poa(subject)
    if poa is null:
        return { allow: false, error: "NO_ACTIVE_POA" }

    // 2. Evaluate policy: does the PoA permit token issuance?
    decision = evaluate_policy(subject, "token_issuance", poa)
    if not decision.allowed:
        return { allow: false, error: decision.reason }

    // 3. Pass the PoA to the after hook via context
    return { allow: true, context: { poa: poa } }
```

F.2 After Hook — Claims Injection (Pseudocode)

```
function after_token_hook(oauth_request, context):
    poa = context.poa
```

```
    // 1. Build gauth claims from the canonical PoA schema
    gauth_claims = {
        version: CURRENT_SPEC_VERSION,
        credential_id: poa.credential_id,
        customer_id: poa.customer_id,
        project_id: poa.project_id,
        scope: {
            governance_profile: poa.scope.governance_profile,
            active_modules: poa.scope.active_modules,
            phase: poa.scope.phase,
            allowed_paths: poa.scope.allowed_paths,
            denied_paths: poa.scope.denied_paths
        },
        scope_checksum: compute_scope_checksum(poa),
        tool_permissions_hash:
sha256(canonical_json(poa.scope.core_verbs)),
        platform_permissions_hash:
sha256(canonical_json(poa.scope.platform_permissions)),
        issued_by: poa.parties.issued_by,
        approval_mode: poa.requirements.approval_mode
    }

    // 2. Build mandate claims if a mandate is active
    mandate_claims = null
    if poa.mandate is not null:
        mandate_claims = {
            mandate_id: poa.mandate.id,
            mandate_status: poa.mandate.status,
            budget: poa.mandate.budget,
            session: poa.mandate.session,
```

```
            agent_capability_hash:
sha256(canonical_json(poa.mandate.capabilities))
        }


    // 3. Return enriched claims for the OAuth engine to
include in the signed JWT
    extra_claims = { gauth: gauth_claims }
    if mandate_claims is not null:
        extra_claims.gauth_mandate = mandate_claims


    return { extra_claims: extra_claims }
```

Appendix G: Open Questions

| # | Question | Context |

|---|----------|---------|

| 1 | **Claims namespace convention** — Flat prefix (`gauth_scope_governance_profile`) vs nested objects (`gauth.scope.governance_profile`). This RFC uses nested objects. | JWT library compatibility |

| 2 | **Token refresh for budget updates** — Re-issuance (new token) vs refresh endpoint (same credential_id). | Session continuity |

| 3 | **Multi-signer encoding for Behörde** — Should the token carry both approver identities? | Audit trail for four-eyes |

| 4 | **Self-hosted JWKS** — Required `.well-known/jwks.json` vs static key distribution. | Deployment simplicity |

| 5 | **PoA VC credential subject DID** — Should agents have persistent DIDs (`did:web`) or ephemeral (`did:key`)? | Identity lifecycle |

| 6 | **SD-JWT claim granularity** — Which claims should be individually disclosable vs always disclosed? | Privacy vs verification needs |

| 7 | **EUDI wallet integration timeline** — When should GAuth issuers register as EAA/QEAA providers? | EU regulatory timeline |

| 8 | **Delegation chain depth limit** — Should the protocol enforce a maximum delegation depth (e.g., 5 levels), or leave this to governance profiles? | Security vs flexibility. Deep chains increase verification cost and attack surface. |

| 9 | **Scope checksum algorithm agility** — Should the spec support algorithm negotiation for `scope_checksum` (e.g., SHA-384, SHA3-256), or lock to SHA-256? | Future-proofing vs simplicity. SHA-256 is sufficient today but may need upgrading. |

| 10 | **VC credential status list partitioning** — Should large issuers maintain multiple status list credentials (partitioned by date, customer, or project), or a single global list? | Scalability. A single list with millions of entries may become unwieldy. |

| 11 | **Canonical JSON serialization standard** — Should the spec reference a specific canonical JSON standard (RFC 8785 — JCS), or define its own? | Interoperability. Different `canonical_json` implementations may produce different outputs. |

| 12 | **Offline verification for budget claims** — How should verifiers handle budget validation when the mandate store is unreachable? Should they accept stale VC budget data with a grace period? | Resilience vs security. Strict stateful validation fails when the store is down. |

| 13 | **GAuth Federation governance** — Who maintains the GAuth Issuer Registry at `gimelfoundation.com/registry`? What are the criteria for inclusion? Should there be a community governance process? | Trust framework. The registry is a trust anchor — its governance model matters. |

Appendix H: Reference Documents

Standards

- [RFC 7519](https://tools.ietf.org/html/rfc7519) — JSON Web Token (JWT)

- [RFC 7515](https://tools.ietf.org/html/rfc7515) — JSON Web Signature (JWS)

- [RFC 7517](https://tools.ietf.org/html/rfc7517) — JSON Web Key (JWK)

- [RFC 8785](https://tools.ietf.org/html/rfc8785) — JSON Canonicalization Scheme (JCS) *(relevant to Section 4.4 scope checksum and Section 4.5 permission hashes — see Open Question #11)*

- [W3C Verifiable Credentials Data Model v2.0](https://www.w3.org/TR/vc-data-model-2.0/)

- [W3C Bitstring Status List v2.0](https://www.w3.org/TR/vc-bitstring-status-list/)

- [OpenID4VCI](https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html) — OpenID for Verifiable Credential Issuance

- [OpenID4VP](https://openid.net/specs/openid-4-verifiable-presentations-1_0.html) — OpenID for Verifiable Presentations

- [SD-JWT](https://datatracker.ietf.org/doc/draft-ietf-oauth-selective-disclosure-jwt/) — Selective Disclosure JWT

- [DID Core](https://www.w3.org/TR/did-core/) — Decentralized Identifiers

- [did:web Method](https://w3c-ccg.github.io/did-method-web/) — Web DID Method Specification

- [eIDAS 2.0 Regulation](https://eur-lex.europa.eu/eli/reg/2024/1183) — EU Digital Identity Framework

- [EUDI Architecture Reference Framework (ARF)](https://github.com/eu-digital-identity-wallet/eudi-doc-architecture-and-reference-framework)

Gimel Foundation RFCs

- GiFo-RFC 0110 — GAuth Protocol Engine

- GiFo-RFC 0111 — GAuth Authorization Framework

- GiFo-RFC 0115 — Three-Layer Capability Framework (Vollmacht)

Disclaimer: ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN “AS IS” BASIS AND THE CONTRIBUTOR, THE ORGANIZATION THEY REPRESENT OR ARE SPONSORED BY (IF ANY), THE GIMEL FOUNDATION, AND ANY APPLICABLE MANAGERS OF ALTERNATE DOCUMENT STREAMS, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

* * *